Woxsen University

School of Technology

B.Tech - Computer Science and Engineering

PROJECT BASED LEARNING REPORT

Title of the Project

(Intelligent Traffic Signal Control Using Fuzzy Logic and Optimization)

Course Code: 24TU03MJM3

Course Name: Foundations of AI and Optimization

Semester: III

Academic Year: 2026-27

Submitted by:

1. D. Ritheek - 25WU0102057
2. M. Srilakshmi Reddy - 25WU0102140
3. B. Deekshan Raj - 25WU0102037
4. P. Priyakaran Reddy - 25WU0101090
5. G. Madhan - 25WU0102344

Section: _____

Under the Guidance of:

Faculty Name: _____

Designation: _____

Department of Computer Science and Engineering

October, 2026

CERTIFICATE

This is to certify that the Project titled **"Intelligent Traffic Signal Control Using Fuzzy Logic and Optimization"** is a bonafide work carried out by the following students of B.Tech - Computer Science and Engineering, during the Academic Year **2026-27**, under my supervision and guidance.

Name	Register Number
D. Ritheek	25WU0102057
M. Srilakshmi Reddy	25WU0102140
B. Deekshan Raj	25WU0102037
P. Priyakaran Reddy	25WU0101090
G. Madhan	25WU0102344

Guide Signature

(Name of the Guide)

Designation

DECLARATION

We hereby declare that the Project titled **"Intelligent Traffic Signal Control Using Fuzzy Logic and Optimization"** submitted by us is an original work carried out under the guidance of the faculty mentioned above.

We confirm that this work has not been submitted in part or full for the award of any degree, diploma, or other academic qualification in this or any other institution.

All sources of information used in this project report have been properly acknowledged and cited as per university referencing standards.

We further declare that the plagiarism percentage is within the permissible limit prescribed by the institution.

Place: _____

Date: October 2026

Signature of the Candidates

Name	Register Number
D. Ritheek	25WU0102057
M. Srilakshmi Reddy	25WU0102140
B. Deekshan Raj	25WU0102037
P. Priyakaran Reddy	25WU0101090
G. Madhan	25WU0102344

ACKNOWLEDGEMENT

We express our sincere gratitude to **Woxsen University** and the **School of Technology** for providing the academic environment and resources needed to complete this project.

We are deeply thankful to our faculty guide for continuous guidance, encouragement and constructive feedback throughout the project. We also thank the faculty of the course **Foundations of AI and Optimization** for the concepts of knowledge representation, genetic algorithms and fuzzy logic that form the basis of this work, and for the reviews that helped us improve the project at each stage.

We thank the Head of the Department of Computer Science and Engineering for the support extended to us. We are grateful to our classmates and friends for their suggestions during discussions, and to our families for their constant support and motivation.

Finally, we thank all the members of the project team for their cooperation, shared effort and commitment, which made it possible to complete this work.

D. Ritheek, M. Srilakshmi Reddy, B. Deekshan Raj, P. Priyakaran Reddy and G. Madhan

ABSTRACT

Most urban traffic signals still run on fixed timers that give every approach the same green time regardless of the vehicles actually waiting. This causes avoidable delay when one road is congested while the cross road is nearly empty. This project designs, tunes and evaluates a Mamdani fuzzy logic controller that sets the green time of a two-phase signalized intersection from live traffic conditions, and uses a genetic algorithm (GA), particle swarm optimization (PSO) and simulated annealing (SA) to tune its membership functions automatically.

Two controller versions were built. Version 1 used two inputs and nine rules. Version 2 uses three inputs (own queue, own waiting time and opposing queue), 27 rules and 12 tunable membership breakpoints. A queueing simulator with Poisson arrivals and time-varying demand was used to generate 120 scenarios, divided into 40 training, 20 validation and 60 test scenarios. Each optimizer received an equal budget of 2000 evaluations, the final controller was selected on the validation set, and the test set was scored only once.

On the 60 unseen test scenarios, version 2 reduced mean delay by 58.5% against a fixed 30-second timer and by 42.4% against version 1 (95% confidence interval 36.2% to 48.5%; 59 wins, 0 losses, 1 tie). A simple actuated baseline, with green time equal to three seconds per queued vehicle, achieved a 57.8% reduction, so version 2 improves on it by only 4.3% (31 wins, 14 losses, 15 ties). The choice of optimizer did not materially change the outcome, and the largest gain came from the controller structure, not from the optimizer. A working prototype with a command-line tool, a local web application and an interactive 3D replay accompanies the study. All data are simulated and only a single two-phase intersection is modelled.

Keywords: Fuzzy logic, Mamdani inference, genetic algorithm, particle swarm optimization, simulated annealing, adaptive traffic signal control, simulation.

TABLE OF CONTENTS

1. INTRODUCTION.....1
1.1 Background.....1
1.2 Motivation and Relevance to a Real-World Problem.....1
1.3 Scope of the Work.....2
1.4 Organization of the Report.....2
2. PROBLEM STATEMENT AND OBJECTIVES.....4
2.1 Problem Definition.....4
2.2 Objectives.....4
2.3 Constraints and Assumptions.....5
3. LITERATURE REVIEW.....6
3.1 Webster: Traffic Signal Settings (1958).....6
3.2 Pappis and Mamdani: A Fuzzy Logic Controller for a Traffic Junction (1977).....6
3.3 Lee: Fuzzy Logic in Control Systems (1990).....6
3.4 SCOOT: A Traffic Responsive Method of Coordinating Signals (1981).....6
3.5 SCATS: The Sydney Co-ordinated Adaptive Traffic System (1982).....7
3.6 Wiering: Multi-Agent Reinforcement Learning for Traffic Light Control (2000). .7
3.7 Abdulhai et al.: Reinforcement Learning for True Adaptive Traffic Signal Control (2003).....7
3.8 Comparison of Related Work.....7
4. METHODOLOGY AND SYSTEM DESIGN.....9
4.1 Overall Architecture.....9
4.2 Fuzzy Controller Design.....9
4.3 Control Cycle and Flowchart.....10
4.4 Traffic Simulator.....11
4.5 Optimization Methods.....12
4.6 Data, Splitting and Evaluation Protocol.....12
4.7 Baselines.....13
4.8 Tools and Technologies.....13
4.9 Data Flow.....14
4.10 Design Decisions and Justification.....14
5. IMPLEMENTATION.....16
5.1 Dataset and Scenario Generation.....16
5.2 Fuzzy Controller Module.....16
5.3 Simulator Module.....17
5.4 Optimizer Module.....17
5.5 Selection and Testing Module.....17
5.6 Prototype Interfaces.....18
5.7 Testing and Reproducibility.....20
5.8 Challenges and Resolutions.....20

6. RESULTS AND DISCUSSION.....21
6.1 Delay Reduction against the Fixed Timer.....21
6.2 Pairwise Comparisons.....22
6.3 Results by Demand Profile.....22
6.4 Queues, Fairness and Throughput.....23
6.5 Effect of the Optimizer and of the Controller Structure.....23
6.6 Robustness beyond the Training Range.....24
6.7 Interpretation.....24
6.8 Comparison with Objectives.....24
6.9 Threats to Validity.....25
7. CONCLUSION AND FUTURE SCOPE.....26
7.1 Summary.....26
7.2 Learning Outcomes.....26
7.3 Future Enhancements.....26
REFERENCES.....27

LIST OF FIGURES

Figure 4.1: Overall system architecture.....9
Figure 4.2: Representative membership functions for queue length (input) and green time (output). The shapes are illustrative; the tuned breakpoints are produced by the optimizer. 10
Figure 4.3: Flowchart of the control cycle.....11
Figure 4.4: Illustrative demand profiles used to generate scenarios (rush-hour ramp, surge, lull)..... 11
Figure 4.5: Genetic algorithm tuning loop around the fuzzy controller.....12
Figure 4.6: Data flow diagram of the system.....14
Figure 5.1: Command-line output showing the recommended green time and the rules that fired.....18
Figure 5.2: Local web application (Traffic Controller Lab): scenario, demand profile, seeds and controller selection.....19
Figure 5.3: 3D replay under the fixed timer (heavy preset, 0.30/0.30 veh/s, constant demand, seed 1). The side panel shows queues and total delay so far.....19
Figure 5.4: The same traffic replayed under the trained fuzzy v2 controller. The panel compares total delay with the fixed timer at the same simulated second.....20
Figure 6.1: Mean delay reduction versus the fixed 30-second timer on 60 test scenarios. 21
Figure 6.2: Improvement of version 2 over version 1 and over the simple rule.....22

LIST OF TABLES

Table 3.1: Comparison of related work.....7
Table 4.1: Summary of the fuzzy controller (version 2).....10
Table 4.2: Division of scenarios.....13
Table 4.3: Tools and technologies.....14
Table 5.1: Main challenges and how they were handled.....20
Table 6.1: Mean delay and reduction relative to the fixed 30-second timer (test set).....21
Table 6.2: Pairwise comparisons on the test set.....22
Table 6.3: Delay reduction relative to the fixed 30 s timer by demand profile (test set)…22
Table 6.4: Other metrics on the test set (means over scenarios).....23
Table 6.5: Optimizer comparison (2000 evaluations each, 3 seeds; fitness lower is better) 23
Table 6.6: Objectives and outcomes.....24

1. INTRODUCTION

1.1 Background

Road traffic is a defining feature of modern cities. As vehicle ownership grows faster than road capacity, congestion has become a routine part of urban life. It lengthens journeys, wastes fuel, increases exhaust emissions and adds to driver stress. A large share of this delay occurs at signalized intersections, where a single timing decision determines how long one direction of traffic moves while the other waits.

Traffic signals have been in use for more than a century, and their control strategies are commonly grouped into three families. **Fixed-time control** repeats a pre-set cycle with constant green durations. It is simple and cheap but cannot respond to changes in demand. **Actuated control** uses detectors to extend or shorten green time according to the vehicles present. **Adaptive control** goes further and continuously adjusts timings using measured demand; well-known examples include the SCOOT and SCATS systems [10], [11]. Many intersections in developing regions still run on fixed timers, either because adaptive systems are expensive or because the timing logic is difficult for local engineers to inspect and trust.

Fuzzy logic, introduced by Zadeh [1], offers a middle path. It represents imprecise human concepts such as a "long queue" or a "short wait" mathematically (an idea developed further in [15]), and it expresses control decisions as readable IF-THEN rules. A fuzzy traffic controller can therefore capture the reasoning of an experienced traffic officer in a form that engineers can read, check and modify. Pappis and Mamdani [7] showed as early as 1977 that a fuzzy controller could operate a traffic junction.

A fuzzy controller is only as good as its parameters. The number, shape and position of the membership functions strongly influence the decisions it makes, and choosing them by hand is largely guesswork. Metaheuristic optimization algorithms such as genetic algorithms [3], [9], particle swarm optimization [4] and simulated annealing [5] can search for good parameter values automatically by evaluating candidate controllers in a simulator. This project combines the two ideas: a fuzzy controller decides the green time, and an optimizer tunes the fuzzy controller.

1.2 Motivation and Relevance to a Real-World Problem

The motivation for this work comes from everyday observation. At many junctions, one approach is packed with vehicles while the opposing approach is empty, yet the signal continues to give both the same green time. Each unnecessary second of green on an empty road is a second of waiting imposed on a queue elsewhere. The cost is borne by every driver, bus passenger and ambulance caught in the queue.

The relevance of the problem can be summarised in four points:

- **Economic:** delay is lost working time and wasted fuel. Even a modest percentage reduction, multiplied over thousands of vehicles per hour, is significant.
- **Environmental:** vehicles idling at red lights emit pollutants without moving. Shorter queues reduce idling.
- **Practical:** a controller that can explain which rules produced a decision is easier for authorities to adopt than an opaque model.
- **Educational:** the problem is a clear, measurable setting in which knowledge representation, fuzzy reasoning and evolutionary optimization, all topics of the course, can be applied together.

The project is also motivated by an honest research question: how much of the benefit of adaptive control comes from a sophisticated controller, and how much comes from simply reacting to queues at all? Chapter 6 answers this question by comparing the fuzzy controller with a deliberately simple adaptive rule.

1.3 Scope of the Work

The scope of this project covers the design, tuning, benchmarking and prototyping of a fuzzy signal controller for a single intersection. Specifically, it includes:

- a Mamdani fuzzy controller for a two-phase (north-south and east-west) intersection;
- a queueing-based traffic simulator with randomly arriving vehicles and time-varying demand;
- tuning of the controller with a genetic algorithm, particle swarm optimization and simulated annealing under equal budgets;
- benchmarking against a fixed 30-second timer and a simple actuated rule on unseen scenarios, with statistical confidence intervals;
- a working prototype consisting of a command-line tool, a local web application and a 3D replay.

The following are outside the scope of the work: real-world traffic counts, multi-intersection networks, turning movements, pedestrian phases, emergency-vehicle priority, and deployment on physical signal hardware. These are discussed as future work in Chapter 7.

1.4 Organization of the Report

The report is organized into seven chapters. **Chapter 1** introduces the background, motivation and scope. **Chapter 2** states the problem, objectives, constraints and assumptions. **Chapter 3** reviews related work and ends with a comparison table. **Chapter 4** describes the methodology and system design, including the architecture, flowcharts and

algorithms. **Chapter 5** describes the implementation of each module of the prototype. **Chapter 6** presents and discusses the results. **Chapter 7** concludes the report, lists the learning outcomes and proposes future enhancements. The references follow in IEEE format.

2. PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem Definition

Consider an isolated intersection controlled by a two-phase signal. In the first phase the north-south approaches receive green; in the second phase the east-west approaches receive green; a short clearance interval separates the two. Vehicles arrive at random, and demand on the two roads changes over time because of rush-hour build-up, short surges and quiet periods. Under fixed-time control the green durations are constants, so the queue on a busy approach can grow while an idle approach holds the green.

The problem addressed in this project is therefore stated as follows: **design a controller that selects the green duration of each phase from the live traffic state, so that the total vehicle delay at the intersection is lower than that of a fixed-time timer, and demonstrate this improvement quantitatively on traffic that was not used to design the controller.**

The problem has two layers. The first is a control problem: given the queue on the approach being served, its waiting time and the queue on the opposing approach, what green time should be chosen? The second is a parameter-tuning problem: the fuzzy controller that answers the first question has many numerical parameters, and good values must be found. The project treats these as a nested problem in which the optimizer searches for the controller parameters that minimize delay in simulation.

2.2 Objectives

The project has four objectives, all of which were completed.

- **Objective 1: Controller.** Design a Mamdani fuzzy controller that maps queue lengths and waiting time to a green duration between 10 and 60 seconds, with a rule base that is readable and explainable.
- **Objective 2: Tuning.** Tune the controller automatically using a genetic algorithm, and compare it with particle swarm optimization and simulated annealing under an equal evaluation budget.
- **Objective 3: Benchmark.** Evaluate the tuned controller against a fixed 30-second timer, a previous controller version and a simple actuated rule, using scenarios that were not used for tuning, and report the results with confidence intervals and win-loss counts.
- **Objective 4: Prototype.** Deliver a working, reproducible prototype that allows the controller to be run, inspected and visually compared with the fixed timer.

Each objective maps to a part of the report: Objective 1 is addressed in Sections 4.2 and 5.2; Objective 2 in Sections 4.4 and 5.4; Objective 3 in Sections 4.5 and 6.1 to 6.4; and Objective 4 in Chapter 5.

2.3 Constraints and Assumptions

The following constraints and assumptions apply to the study.

- **Simulated data.** No real traffic data were available. All scenarios are generated by a simulator, so the results measure performance in the simulated environment and should not be read as field measurements.
- **Single intersection.** The model contains one intersection with two phases. Interaction between neighbouring junctions is not considered.
- **Arrival process.** Vehicle arrivals follow a Poisson process whose rate, piecewise linear in time, is set by one of six demand profiles. One simulated run lasts 3600 seconds.
- **Discharge model.** Queued vehicles discharge at a saturation flow of 0.5 vehicles per second, and a 3-second clearance interval separates the phases.
- **Green-time range.** The controller output is restricted to 10 to 60 seconds so that no phase is starved or held unreasonably long.
- **Equal budget.** All optimizers receive the same budget of 2000 evaluations so that the comparison is fair.
- **Single evaluation of the test set.** The test scenarios are used once, after the final controller has been chosen on validation data, to avoid optimistic bias.
- **Reproducibility.** All random processes use fixed seeds so that every reported number can be regenerated.

3. LITERATURE REVIEW

This chapter reviews seven works that relate to the project: classical signal timing, fuzzy traffic control, two operational adaptive systems, two reinforcement-learning approaches and the foundational work on fuzzy control. They are compared at the end of the chapter.

3.1 Webster: Traffic Signal Settings (1958)

Webster [8] produced the classical analysis of fixed-time signal settings. By studying delay at signalized intersections, he derived expressions for the cycle length that minimizes average delay and a rule for dividing green time among phases in proportion to demand. His work remains the basis of many fixed-time plans. The limitation relevant to this project is that the plan is computed for an assumed constant demand; once the plan is set, it cannot respond to short-term variations in traffic. This is the behaviour that the fixed 30-second timer in our experiments represents.

3.2 Pappis and Mamdani: A Fuzzy Logic Controller for a Traffic Junction (1977)

Pappis and Mamdani [7] applied fuzzy control to a traffic junction, using rules written in linguistic form to decide whether to extend or end the current green. The paper is important because it demonstrated that the rule-based control ideas of Mamdani and Assilian [2] were applicable to traffic. Its controller used a hand-designed rule base and hand-chosen membership functions; there was no automatic tuning. The present project builds on this idea and adds an optimizer to choose the membership parameters, and a formal train-validation-test evaluation.

3.3 Lee: Fuzzy Logic in Control Systems (1990)

Lee [6] published a widely cited two-part survey of fuzzy logic controllers. It describes the structure of a fuzzy controller in detail: fuzzification, knowledge base, inference engine and defuzzification, along with alternatives for each stage. This tutorial treatment guided the structure of our controller, in particular the choice of centroid defuzzification and the organization of the rule base. It also noted the lack of systematic methods for choosing membership functions, which is the gap that the optimization part of our work addresses.

3.4 SCOOT: A Traffic Responsive Method of Coordinating Signals (1981)

SCOOT [10] was developed by the Transport Research Laboratory in the United Kingdom. It uses detector data to build a model of queues approaching a set of coordinated signals and continually adjusts the cycle, split and offset in small steps. SCOOT showed that on-line adaptive control can reduce delay compared with fixed plans across a network. It is a network-level system that requires extensive detector infrastructure and a central computer, which is beyond the scope of an isolated-junction study, but it motivates the use of live queue information as the controller input.

3.5 SCATS: The Sydney Co-ordinated Adaptive Traffic System (1982)

SCATS [11] was developed in Sydney and is used in many cities around the world. It selects among pre-defined plans and adjusts splits according to measured degree of saturation at the stop line. Like SCOOT, it demonstrates the value of responding to measured demand. SCATS uses a plan-selection hierarchy rather than learning or rule-based reasoning, so its decisions are less easily expressed as simple human-readable rules than those of a fuzzy controller.

3.6 Wiering: Multi-Agent Reinforcement Learning for Traffic Light Control (2000)

Wiering [13] proposed that traffic lights could learn their control policy through reinforcement learning, using a simulator to evaluate the effect of decisions on waiting times of vehicles. The approach does not require a hand-written rule base and can discover non-obvious strategies. The trade-offs are that learning requires many simulation episodes, the learned policy is usually opaque, and results are sensitive to the design of the state and reward. Our work keeps a transparent rule base and uses optimization only to tune parameters.

3.7 Abdulhai et al.: Reinforcement Learning for True Adaptive Traffic Signal Control (2003)

Abdulhai, Pringle and Karakoulas [12] studied Q-learning for adaptive signal control of an intersection and examined how problem formulation affects learning. They showed that reinforcement learning is a plausible route to adaptive control, while also highlighting practical issues such as the size of the state space and the need for careful reward design. For this project, the paper is a reminder that learning-based methods are a serious alternative and that any claim of advantage should be tested against simple baselines, which is why we include a simple actuated rule in our benchmark.

3.8 Comparison of Related Work

Table 3.1 compares the reviewed works on their approach, adaptivity, explainability and relation to this project.

Table 3.1: Comparison of related work

Work	Approach	Adapts to live demand	Readable rules	Relation to this project
Webster [8]	Analytical fixed-time plan	No	Not applicable	Basis of the fixed-timer baseline
Pappis and Mamdani [7]	Hand-built fuzzy controller	Yes	Yes	Starting point; we add automatic tuning

Work	Approach	Adapts to live demand	Readable rules	Relation to this project
Lee [6]	Survey of fuzzy controller design	Not applicable	Yes	Guides the controller structure
SCOOT [10]	Network model-based adaptive control	Yes	No	Motivates use of live queues
SCATS [11]	Plan selection and split adjustment	Yes	Partly	Motivates adaptive control
Wiering [13]	Multi-agent reinforcement learning	Yes	No	Learning alternative
Abdulhai et al. [12]	Q-learning for one intersection	Yes	No	Reminder to test simple baselines
This project	Fuzzy controller tuned by GA / PSO / SA	Yes	Yes	Explainable and optimizer-tuned

The review shows a clear gap. Fuzzy controllers are explainable but have traditionally been tuned by hand, while learning-based methods adapt well but are hard to inspect. This project places an optimizer around an explainable fuzzy controller, and evaluates it against simple baselines on unseen scenarios.

4. METHODOLOGY AND SYSTEM DESIGN

4.1 Overall Architecture

The system consists of four cooperating parts: the fuzzy controller, the traffic simulator, the optimizer and the prototype front end. The simulator generates traffic and measures delay for any controller. The fuzzy controller reads the state of the intersection and decides the green time. The optimizer searches for the controller parameters that give the lowest delay. The prototype exposes the tuned controller to the user. Figure 4.1 shows how these parts connect.

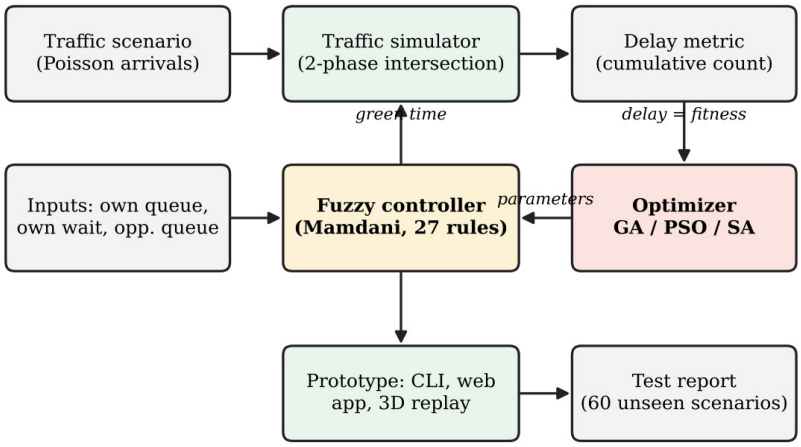


Figure 4.1: Overall system architecture

An important design decision is that the optimizer does not replace the fuzzy logic. It wraps around it. The fitness value that the optimizer receives is the delay obtained by running the fuzzy controller in the simulator, so the optimizer improves the controller without changing its structure or making its rules less readable.

4.2 Fuzzy Controller Design

The controller is a Mamdani-type fuzzy inference system [2], [6], [16]. Version 2 takes three crisp inputs: the queue on the approach about to receive green, its waiting time (the seconds since that approach last had green), and the queue on the opposing approach. The opposing queue is the main addition over version 1 and gives the controller awareness of the competing demand.

Fuzzification. Each input is converted to degrees of membership in three linguistic sets: low, medium and high for the two queue inputs, and short, moderate and long for the waiting time. The membership functions are piecewise-linear, and their breakpoints are parameters that the optimizer tunes. Figure 4.2 shows representative membership functions for the queue input and for the output.

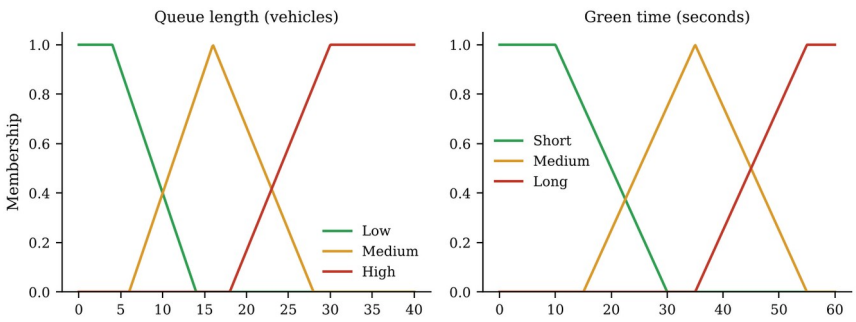


Figure 4.2: Representative membership functions for queue length (input) and green time (output). The shapes are illustrative; the tuned breakpoints are produced by the optimizer.

Rule base. With three inputs and three sets each, there are 3 x 3 x 3 = 27 combinations, and one IF-THEN rule is defined for each. The rules express the intuition of a traffic officer. For example: IF own queue is high AND opposing queue is low THEN green time is long. IF own queue is low AND opposing queue is high THEN green time is short. This rule base is a form of knowledge representation, the subject of the first weeks of the course.

Inference and defuzzification. For each rule, the firing strength is computed from the membership degrees of its conditions using the minimum operator, and the output sets are clipped accordingly. The clipped sets are aggregated by the maximum operator. The centroid of the aggregated set, defined over an output range of 0 to 80 seconds, is taken as the crisp green time and is then clamped to the physical range of 10 to 60 seconds. The prototype records which rules fired (Figure 5.1 shows a real example), so each decision can be explained.

Table 4.1: Summary of the fuzzy controller (version 2)

Element	Specification
Inputs	Own queue, own waiting time, opposing queue
Linguistic sets per input	Queues: low, medium, high; waiting: short, moderate, long
Number of rules	27 (3 x 3 x 3)
Inference method	Mamdani (min for AND, clipping, max aggregation)
Defuzzification	Centroid
Output	Green time, 0 to 80 s universe, clamped to 10 to 60 s
Tunable parameters	12 ordered membership breakpoints (a repair step keeps every candidate valid)

4.3 Control Cycle and Flowchart

The controller operates once per phase. At the start of each phase it reads the detector values, computes the green time through fuzzy inference, applies the green, allows the clearance interval and then switches to the other phase. Figure 4.3 shows this cycle.

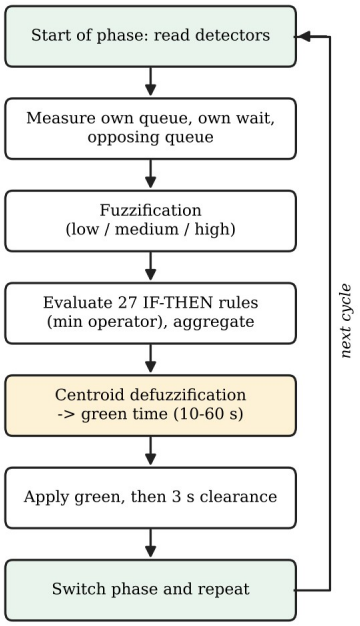


Figure 4.3: Flowchart of the control cycle

4.4 Traffic Simulator

To score controllers, a queueing simulator of a two-phase intersection was written. Vehicles arrive at each approach according to a Poisson process whose rate varies with time, so that rush-hour ramps, surges and lulls can be represented (Figure 4.4). During green, the queue discharges at a saturation flow of 0.5 vehicles per second (saturation flow is a standard concept in signal design [19]); a 3-second clearance interval separates the phases. Each run lasts 3600 seconds. Delay is measured as the sum, over every second, of the number of queued vehicles, divided by the number of vehicles that arrived in the hour; this gives the mean delay per vehicle in seconds and is equivalent to the cumulative-count method. The same definition was used for version 1. The simulator is deterministic for a given seed. Version 2 of the simulator reproduces version 1 exactly on constant demand (maximum relative difference 0.0).

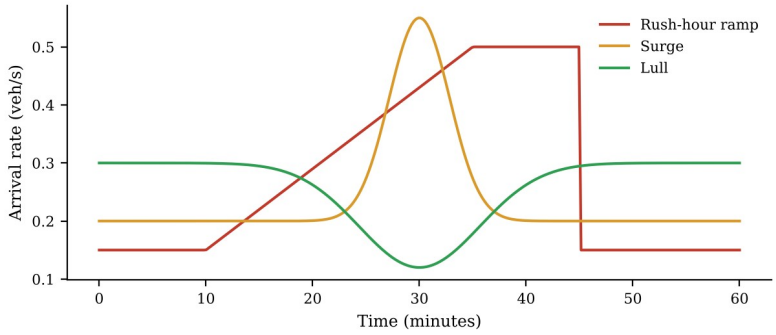


Figure 4.4: Illustrative demand profiles used to generate scenarios (rush-hour ramp, surge, lull)

4.5 Optimization Methods

A candidate solution is a vector containing all the membership breakpoints of the controller. To evaluate it, the vector is converted into a fuzzy controller, which is run in the simulator on the training scenarios. The fitness is the mean delay divided by the delay of the best fixed timer on the same run, averaged over the training scenarios (lower is better, and below 1 means better than the best fixed timer). Dividing by the fixed-timer delay lets a 5% change count equally in light and in heavy traffic; version 1, which used raw delay, was dominated by the heavy scenarios. Three optimizers were compared.

- **Genetic algorithm (GA)** [3], [9], [14]. A population of candidate vectors is evolved over generations. The fitter candidates are selected as parents, new candidates are produced by crossover (mixing two parents) and mutation (small random changes), and the cycle repeats.
- **Particle swarm optimization (PSO)** [4], [17]. Each candidate is a particle that moves through the search space. Its velocity is influenced by its own best position and the best position of the swarm.
- **Simulated annealing (SA)** [5]. A single candidate is modified by small random moves. Worse moves are accepted with a probability that decreases as the search "cools", which helps the search escape local optima.

All three treat the simulator as a black box, which is appropriate because the delay surface has no analytical gradient. Each optimizer received the same budget of 2000 evaluations, and each was run with three seeds on each of three controller structures (version 1 re-trained, version 2 with 12 breakpoints, and version 2 with 12 breakpoints plus 27 rule weights), giving 27 optimization runs in total. Figure 4.5 shows the GA tuning loop.

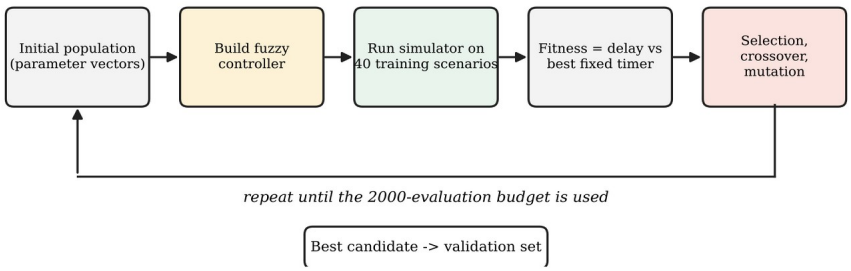


Figure 4.5: Genetic algorithm tuning loop around the fuzzy controller

4.6 Data, Splitting and Evaluation Protocol

No real-world traffic dataset was used. Instead, 120 scenarios were generated across six demand profiles: constant, rush-hour ramp, surge, one-sided surge, lull, and balanced-then-skewed. They were split into training, validation and test sets as shown in Table 4.2. The training scenarios are used for fitness evaluation, the validation scenarios for choosing the

final controller among the candidates, and the test scenarios are scored once. Each test scenario is run with three random seeds.

Table 4.2: Division of scenarios

Set	Scenarios	Purpose
Training	40	Fitness evaluation during optimization
Validation	20	Selection of the final controller
Test	60	Final scoring, performed once

Results are reported as the percentage reduction in mean delay relative to the fixed 30-second timer. Uncertainty is quantified by 95% confidence intervals obtained by bootstrap resampling over scenarios (10,000 resamples) [18]. Controllers are also compared by paired win, loss and tie counts across scenarios, where a tie means a difference of less than 1%, a threshold fixed before the test set was opened. The promotion rule was also fixed in advance: version 2 is adopted only if its mean paired reduction against version 1 is positive and the confidence interval excludes zero.

4.7 Baselines

- **Fixed 30-second timer.** Each phase receives 30 seconds of green regardless of demand. This is the reference for all percentage reductions. Two other fixed timers were also tuned on the training set only (29 s and 56 s).
- **Fuzzy version 1 with GA.** The earlier controller with two inputs, nine rules and three tunable membership centres.
- **Simple actuated rule.** Green time equal to three seconds multiplied by the queue on the served approach, clipped to 10 to 60 seconds. This one-line rule is a strong, easily understood adaptive baseline.
- **Pressure-split and longest-queue-first rules.** Two further adaptive heuristics, tuned on the training set only.

4.8 Tools and Technologies

The project was implemented with freely available tools, summarised in Table 4.3.

Table 4.3: Tools and technologies

Tool	Use in the project
Python 3 [20]	Controller, simulator, optimizers, command-line tool, web server and tests
NumPy [21]	Numerical computation for membership functions and statistics
HTML, CSS and JavaScript	Local web application and the 3D replay in the browser
Unit-test suite	23 automated tests covering the controller and simulator
Fixed random seeds	Reproducibility of every reported result

4.9 Data Flow

Figure 4.6 shows the data flow of the system. Traffic scenarios supply queue information to the fuzzy controller, which returns green times to the simulator. The simulator returns state information to the controller and a delay value to the optimizer. The optimizer produces new parameters for the controller and stores the best ones, and the results are written to a report.

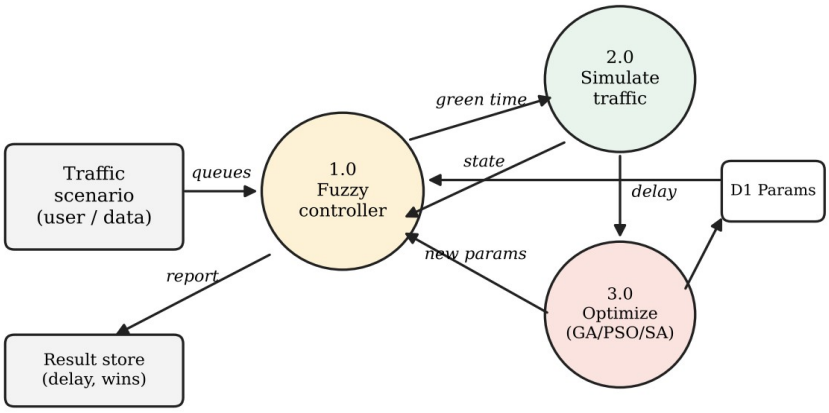


Figure 4.6: Data flow diagram of the system

4.10 Design Decisions and Justification

Several design choices shaped the system, and each was made for a specific reason.

- **Why fuzzy logic.** Traffic conditions are graded rather than sharp, and the decision logic must be readable. A fuzzy rule base provides both. A learned black-box policy was considered, but it would not have allowed each decision to be traced to rules.
- **Why three linguistic sets per input.** Three sets (low, medium, high) keep the rule base to 27 rules, which is complete enough to cover every combination while remaining small enough to read and verify by hand.
- **Why an optimizer around the controller.** Hand-picked membership functions are guesses. Treating the breakpoints as parameters and letting an optimizer minimize

simulated delay replaces guesswork with measurement, without altering the readable rule structure.

- **Why three optimizers with equal budgets.** Comparing GA, PSO and SA under the same 2000 evaluations tests whether the conclusion depends on the search method, and prevents one method from winning merely by receiving more computation.
- **Why train, validation and test sets.** Tuning on one set of scenarios and reporting results on the same set would overstate performance. Selecting on validation data and scoring the test data once gives an honest estimate for traffic the controller has not seen.
- **Why a simple rule as a baseline.** A fixed timer is an easy opponent. Including an adaptive rule that anyone can write in one line shows how much of the improvement is due to the fuzzy design itself.

5. IMPLEMENTATION

This chapter describes how each module of the system was implemented. The code is organised in a version 2 folder of the project, containing the controller, the simulator, the optimizers, a command-line tool (predict.py), a local web server (server.py), a single command that reproduces all results (run_all.py), a tests folder and the 3D replay page (replay_3d_v2.html).

5.1 Dataset and Scenario Generation

The project uses simulated traffic only. A scenario is defined by an arrival-rate profile for each of the two approaches over the simulation horizon. Six demand profiles were defined (constant, rush-hour ramp, surge, one-sided surge, lull, and balanced-then-skewed), and 120 scenarios were generated from them with fixed generator seeds, so that the training, validation and test sets are disjoint. This disjointness is checked in the code and by a unit test. The scenarios were then assigned to the training, validation and test sets as described in Section 4.6. Generating the data in this way gives much wider coverage than the three hand-picked scenarios used in version 1, and allows the test set to contain traffic patterns that were never seen during tuning.

5.2 Fuzzy Controller Module

The controller module exposes a function that takes the three crisp inputs and a parameter vector, and returns the green time together with the list of rules that fired. Membership degrees are computed from the breakpoints in the parameter vector, rule strengths are computed with the minimum operator, and the centroid is obtained by numerical integration over a discretised output range. Keeping the parameter vector separate from the controller logic allows the optimizer to construct a new controller for every candidate without changing any code. The pseudo code is given below.

```
function FUZZY_GREEN(own_q, own_wait, opp_q, params):
    mu_q   <- memberships(own_q,   params.queue)      # low, med, high
    mu_w   <- memberships(own_wait, params.wait)
    mu_o   <- memberships(opp_q,   params.opposing)
    aggregated <- empty fuzzy set over [10, 60]
    fired <- []
    for each rule r in RULES (27):
        strength <- min(mu_q[r.q], mu_w[r.w], mu_o[r.o])
        if strength > 0:
            fired.append(r)
            clipped <- clip(r.output_set, strength)
            aggregated <- max(aggregated, clipped)
    green <- centroid(aggregated)
    return clamp(green, 10, 60), fired
```

5.3 Simulator Module

The simulator advances time in small steps. At each step, random arrivals are added to the queues according to the current arrival rate; during green, vehicles leave the served queue at the saturation flow rate; at the end of a green period, a 3-second clearance interval is applied and the phase is switched. The controller is called at the start of every phase to obtain the next green time. Delay is accumulated using the cumulative-count method, which makes it independent of how individual vehicles are tracked. The simulator accepts any controller as a function, so the fixed timer, the actuated rule and the fuzzy controllers are all evaluated by exactly the same code.

```
function SIMULATE(controller, scenario, seed):
    queues <- [0, 0];  phase <- 0;  t <- 0;  total_delay <- 0
    while t < scenario.horizon:
        green <- controller(queues, waits, phase)
        for each second in green:
            queues <- queues + poisson_arrivals(scenario, t, seed)
            queues[phase] <- queues[phase] - min(queues[phase], 0.5)
            total_delay <- total_delay + sum(queues)
            t <- t + 1
        t <- t + 3                      # clearance interval
        phase <- 1 - phase
    return total_delay
```

5.4 Optimizer Module

The optimizer module contains the GA, PSO and SA implementations. They share a common interface: each receives an objective function, a search range for every parameter and an evaluation budget, and returns the best parameter vector found. The objective function builds a fuzzy controller from the vector and returns its normalised mean delay on the training scenarios. Because the interface is shared, the three optimizers could be compared fairly with an identical budget of 2000 evaluations. The pseudo code for the GA is given below.

```
function GA(objective, bounds, budget):
    pop <- random_population(bounds)
    evaluations <- 0
    while evaluations < budget:
        fitness <- [objective(c) for c in pop]; evaluations += len(pop)
        parents <- select_best(pop, fitness)
        children <- crossover(parents) then mutate(children)
        pop <- parents + children
    return best(pop)
```

5.5 Selection and Testing Module

After optimization, the candidates from all runs (18 in total) are scored on the validation scenarios, and the one with the lowest validation score is selected as the final controller. This was a PSO run with seed 2 on the 12-breakpoint structure. The final controller is then run once on the 60 test scenarios, each with three seeds. For each scenario, the percentage

reduction relative to the fixed timer is computed. Mean reductions, bootstrap confidence intervals and paired win-loss-tie counts are then calculated. Keeping selection and testing separate prevents the optimizer from fitting the test data.

5.6 Prototype Interfaces

The prototype has three interfaces, each aimed at a different use.

- **Command-line tool (predict.py).** Takes the three input values and prints the recommended green time together with the rules that fired and their strengths. This is the quickest way to inspect and explain a single decision.
- **Local web application (server.py).** Started with one command and opened in the browser at localhost on port 8000. The user selects a scenario and a controller, the real Python simulator runs behind the page, and the results are displayed. It works fully offline.
- **3D replay (replay_3d_v2.html).** A browser-based 3D view of the intersection in which the same traffic is replayed under the fixed timer and under the fuzzy controller. The user can switch between the two at the same simulated second, which makes the effect of adaptive control visible.

Figures 5.1 to 5.4 show the prototype in use. Figure 5.2 shows the web application, where the scenario, demand profile, random seeds and controller are chosen. Figures 5.3 and 5.4 show the 3D replay of the same traffic under the fixed timer and under the trained fuzzy v2 controller. In this single heavy, constant-demand demonstration run (0.30 vehicles per second on each road, seed 1), the side panel reported total delay so far to be 11.1% to 11.9% lower under fuzzy v2 at the moments captured. This demand exceeds the capacity of the modelled intersection (about 0.45 vehicles per second in total), so every controller builds a long queue and the differences are small. This is one demonstration scenario and must not be confused with the test-set results of Chapter 6.

```
$ python3 predict.py --queue 30 --wait 60 --opp 8
controller: fuzzy-v2 (trained)
inputs    : own queue 30 veh | waiting 60 s | opposing queue 8 veh
  own_queue       low=0.00  med=0.00  high=1.00
  waiting         short=1.00  moderate=0.00  long=0.00
  opposing_queue  low=0.00  med=0.93  high=0.07
rules fired (strongest first):
  R20  strength 0.927  IF own_queue is high AND waiting is short
       AND opposing_queue is med THEN green is long
  R21  strength 0.073  IF own_queue is high AND waiting is short
       AND opposing_queue is high THEN green is long
output strengths: {'short': 0.0, 'medium': 0.0, 'long': 0.9266}
RECOMMENDED GREEN: 57.8 s   (defuzzified 57.78 s, clamped to 10-60 s)
```

Figure 5.1: Command-line output showing the recommended green time and the rules that fired

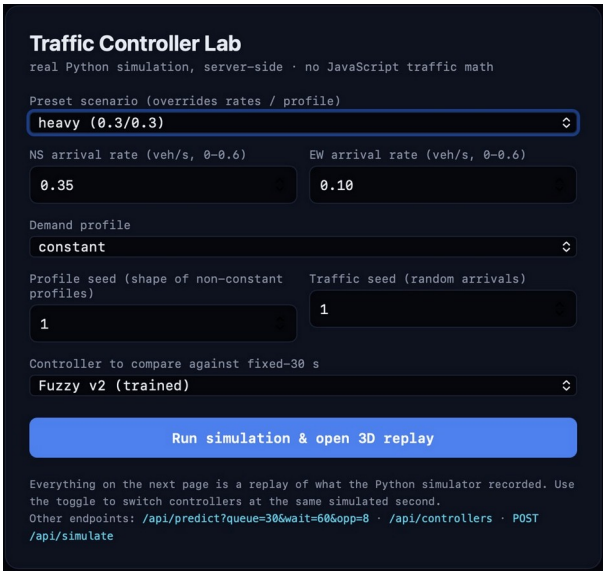


Figure 5.2: Local web application (Traffic Controller Lab): scenario, demand profile, seeds and controller selection



Figure 5.3: 3D replay under the fixed timer (heavy preset, 0.30/0.30 veh/s, constant demand, seed 1). The side panel shows queues and total delay so far



Figure 5.4: The same traffic replayed under the trained fuzzy v2 controller. The panel compares total delay with the fixed timer at the same simulated second

5.7 Testing and Reproducibility

Twenty-three unit tests check the main behaviour of the controller and the simulator, for example that the green time always lies between 10 and 60 seconds and that identical seeds give identical delays. A single command (run_all.py) reproduces every reported result from fixed seeds, from scenario generation through optimization to the final test-set scoring. This was important because several earlier numbers from a preliminary five-scenario, single-seed evaluation could not be reproduced under the stricter protocol; only numbers obtained from the final protocol are reported in this document.

5.8 Challenges and Resolutions

Table 5.1: Main challenges and how they were handled

Challenge	Resolution
Version 1 could not see the opposing road	Added the opposing queue as a third input and expanded the rule base to 27 rules
Too few evaluation scenarios	Generated 120 scenarios across six demand profiles with a train, validation and test split
Risk of optimistic results	Selected the final controller on validation data and scored the test set only once
Unfair comparison of optimizers	Gave GA, PSO and SA an identical budget of 2000 evaluations
Possibility of a weak baseline	Added a simple actuated rule as a stronger benchmark

6. RESULTS AND DISCUSSION

6.1 Delay Reduction against the Fixed Timer

All results in this chapter were measured on the 60 test scenarios (three fresh seeds each), which were not used for tuning or for controller selection and were evaluated exactly once. Table 6.1 and Figure 6.1 show the mean delay and the mean reduction relative to the fixed 30-second timer, with 95% bootstrap confidence intervals.

Table 6.1: Mean delay and reduction relative to the fixed 30-second timer (test set)

Controller	Mean delay (s)	Reduction vs fixed 30 s (95% CI)
Fixed 30 s timer	173.5	0.0%
Best fixed timer (56 s)	160.4	-6.9% (-13.9 to -0.6)
Longest-queue-first	87.8	+23.1% (+9.5 to +36.3)
Pressure-split	88.4	+24.9% (+11.3 to +37.7)
Fuzzy v1, hand-set	121.8	+29.4% (+24.8 to +34.3)
Fuzzy v1 + GA	105.9	+34.9% (+29.6 to +40.3)
Fuzzy v2, hand-set	85.4	+47.8% (+41.4 to +54.1)
Simple actuated rule	74.6	+57.8% (+51.3 to +64.2)
Fuzzy v2 + PSO (proposed)	**71.9**	**+58.5% (+51.7 to +65.1)**

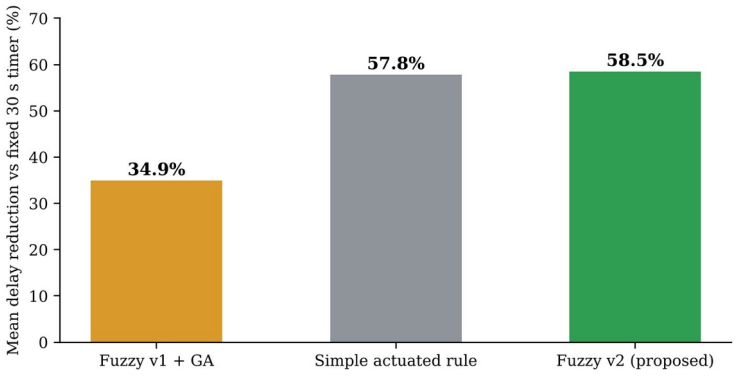


Figure 6.1: Mean delay reduction versus the fixed 30-second timer on 60 test scenarios

The proposed version 2 controller reduces mean delay from 173.5 s to 71.9 s, a reduction of 58.5%. It never lost to the fixed timer: it won in all 60 scenarios, and its worst single scenario was still 6.3% better. Version 1 with GA reached 34.9%. The simple actuated rule, at 57.8%, is very close to version 2. The best fixed timer chosen on raw training delay (56 s) is worse than the 30-second timer on the test set, because it performs poorly in lulls and surges.

6.2 Pairwise Comparisons

Table 6.2: Pairwise comparisons on the test set

Comparison	Improvement	Details
Version 2 versus version 1 + GA	42.4%	95% CI 36.2% to 48.5%; 59 wins, 0 losses, 1 tie
Version 2 versus simple actuated rule	4.3%	95% CI 2.4% to 6.5%; 31 wins, 14 losses, 15 ties

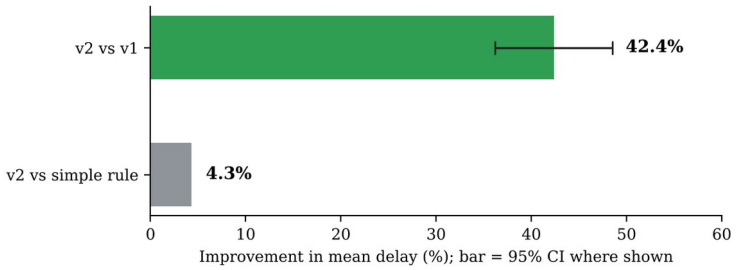


Figure 6.2: Improvement of version 2 over version 1 and over the simple rule

The promotion rule was met: the mean paired reduction against version 1 is positive and its interval excludes zero, so version 2 is adopted. Version 2 beats version 1 in 59 of 60 scenarios. Against the simple actuated rule the improvement of 4.3% is statistically distinguishable from zero but practically small, and version 2 loses in 14 of the 60 scenarios, so it does not dominate the simple rule. A notable finding is that version 1 with GA, at 34.9%, is far below this one-line rule: version 1 beat the fixed timer it was compared against, but not a trivial adaptive baseline.

6.3 Results by Demand Profile

Table 6.3: Delay reduction relative to the fixed 30 s timer by demand profile (test set)

Controller	Bal. then skewed	Const.	Lull	One-sided surge	Rush ramp	Surge
Best fixed (56 s)	+4.8%	+6.2%	-56.9%	+1.7%	+2.1%	-15.9%
Fuzzy v1 + GA	+44.8%	+42.3%	+11.3%	+47.6%	+33.7%	+22.2%
Actuated rule	+83.0%	+57.9%	+42.5%	+74.0%	+49.7%	+42.6%
Fuzzy v2 + PSO	**+84.8%**	**+59.5%**	**+41.6%**	**+75.3%**	**+49.7%**	**+42.0%**

Version 2 and the actuated rule are within about two percentage points of each other on every profile. Version 1 is weakest in lulls (11.3%) and surges (22.2%), where the fixed-timer-based training did not prepare it for changing demand.

6.4 Queues, Fairness and Throughput

Table 6.4: Other metrics on the test set (means over scenarios)

Controller	p95 queue (veh)	Max queue (veh)	Worst-direction delay (s)	Served / arrived
Fixed 30 s	168.8	183.0	223.5	0.910
Fuzzy v1 + GA	108.8	119.7	126.9	0.953
Actuated rule	82.9	92.0	88.0	0.969
Fuzzy v2 + PSO	**80.3**	**89.4**	**82.8**	**0.971**

Version 2 has the shortest queues and the lowest delay for the worst-served direction, which indicates that it is slightly fairer than the other controllers as well as faster on average. These margins over the actuated rule are small.

6.5 Effect of the Optimizer and of the Controller Structure

Table 6.5: Optimizer comparison (2000 evaluations each, 3 seeds; fitness lower is better)

Structure	Optimizer	Train fitness (mean +/- sd)	Validation fitness (mean)	Evaluations to 95% of gain
Fuzzy v1 re-trained	GA	0.6391 +/- 0.0001	0.7135	383
Fuzzy v1 re-trained	PSO	0.6392 +/- 0.0008	0.7048	367
Fuzzy v1 re-trained	SA	0.6395 +/- 0.0002	0.7141	713
Fuzzy v2 (12 breakpoints)	GA	0.4143 +/- 0.0019	0.5103	637
Fuzzy v2 (12 breakpoints)	PSO	0.4186 +/- 0.0047	0.5156	590
Fuzzy v2 (12 breakpoints)	SA	0.4189 +/- 0.0017	0.5151	723

The three optimizers end within the seed-to-seed spread of each other. PSO converged fastest and SA slowest, but all reach the same plateau. Adding 27 rule weights (39 parameters in total) did not help and converged more slowly. The structure of the controller matters far more: re-training the version 1 controller on the new data gives a training fitness of 0.639, while version 2 reaches about 0.414. The choice of PSO seed 2 as the official controller rests on a validation margin of only 0.001 to 0.002, which is within noise, so the selection is nominal and not a discovery. All six version 2 candidates, if evaluated on the test set, would have passed the promotion rule (+40.9% to +42.4% against version 1).

6.6 Robustness beyond the Training Range

As an additional check not used for any decision, 24 constant-demand scenarios with arrival rates up to 0.55 vehicles per second on each road (beyond the 0.40 limit of the generators) were run with fresh seeds. Against a fixed-timer delay of 552.8 s, version 2 reduced delay by 33.1%, the actuated rule by 32.0%, the hand-set version 2 by 27.4% and version 1 with GA by 20.5% (the intervals of version 2 and the actuated rule overlap). All controllers degrade when the intersection is over-saturated, but none breaks down.

6.7 Interpretation

- **Adaptive control is clearly valuable.** The adaptive controllers cut delay by 35% to 58% compared with the fixed timer. Most of the benefit comes from reacting to the queues at all.
- **Version 2 is a real improvement over version 1.** Giving the controller visibility of the opposing road, and normalising the fitness, accounts for the gain of 42.4%.
- **The advantage over a good simple rule is small.** A one-parameter rule gets nearly all of the way. The honest reading is that the fuzzy controller matches the best simple adaptive rule and edges it slightly, while also being explainable through the rules that fired.
- **The waiting-time input is rarely used.** The tuned breakpoints for waiting time moved to between 71 and 104 seconds, so this input seldom fires and the controller relies mainly on the two queues. This is consistent with the actuated rule, which uses only the queue, doing almost as well.
- **Explainability is the practical strength.** Each decision can be traced to the rules that fired, which is helpful for validation and for communicating with engineers.

6.8 Comparison with Objectives

Table 6.6: Objectives and outcomes

Objective	Outcome
1. Controller	Mamdani controller with three inputs and 27 rules built; output clamped to 10 to 60 seconds; rules that fired are reported for each decision
2. Tuning	GA, PSO and SA compared under equal 2000-evaluation budgets (27 runs); the optimizer choice did not materially change results
3. Benchmark	Evaluated once on 60 unseen scenarios against several baselines; 58.5% reduction versus the fixed timer, 42.4% versus version 1, 4.3% versus the simple rule
4. Prototype	Command-line tool, local web application and 3D replay delivered; 23 tests pass; one command reproduces all results

6.9 Threats to Validity

The results come from simulation, so their validity depends on how well the simulator represents real intersections. Arrivals are Poisson with piecewise-linear rates, and there are no platoons, turning movements, pedestrians or sensor noise. Controllers choose only the duration of the next green in a strictly alternating two-phase cycle. The confidence intervals are bootstrap intervals over scenarios drawn from our own generator, so they show that the result is stable across that distribution, not that it transfers to real roads. In over-saturated scenarios the delay measure depends on the one-hour horizon, because vehicles still queued at the end contribute only the waiting so far. The results should therefore be read as a comparison of control strategies under identical simulated conditions, not as a prediction of field performance.

7. CONCLUSION AND FUTURE SCOPE

7.1 Summary

This project designed, tuned and evaluated a Mamdani fuzzy traffic signal controller. Version 2 uses three inputs and 27 IF-THEN rules to select a green time of 10 to 60 seconds, and its membership functions are tuned by optimization within a simulator. On 60 unseen simulated scenarios it reduced delay by 58.5% against a fixed 30-second timer and by 42.4% against the earlier version. It also matched a simple actuated rule, improving on it by only 4.3%. The optimizer used (GA, PSO or SA) did not matter, and the largest gain came from the controller structure, namely adding the opposing queue to the inputs. The complete prototype includes a command-line tool, a local web application and a 3D replay, and all results can be reproduced by a single command. All four objectives were achieved.

7.2 Learning Outcomes

- Applied knowledge representation by building and reasoning over a 27-rule fuzzy rule base.
- Applied fuzzy inference with centroid defuzzification to a real-world control problem.
- Applied genetic algorithms, particle swarm optimization and simulated annealing to tune a controller and compared them fairly.
- Learned to separate training, validation and test data, and to report results with confidence intervals and win-loss counts.
- Learned the value of simple baselines: a one-line rule was almost as good as the more elaborate controller, and it exposed that the first version had been compared only against a weak opponent.
- Practised building a reproducible software prototype with tests.

7.3 Future Enhancements

- **Real data.** Validate and tune the controller on measured traffic counts from real intersections.
- **Richer controller inputs.** Add inputs such as queue trend and time since last green, and test whether they widen the margin over the simple rule.
- **Pedestrians and emergency vehicles.** Add pedestrian crossing requests with walk times, and an override that gives green to an approaching emergency vehicle.
- **Multiple phases and intersections.** Extend to turning movements and to a corridor of coordinated junctions with a green wave.
- **Hardware deployment.** Test the controller with real detectors and signal controllers.

REFERENCES

[1] L. A. Zadeh, "Fuzzy sets," Information and Control, vol. 8, no. 3, pp. 338-353, 1965.

[2] E. H. Mamdani and S. Assilian, "An experiment in linguistic synthesis with a fuzzy logic controller," International Journal of Man-Machine Studies, vol. 7, no. 1, pp. 1-13, 1975.

[3] J. H. Holland, Adaptation in Natural and Artificial Systems. Ann Arbor, MI, USA: University of Michigan Press, 1975.

[4] J. Kennedy and R. Eberhart, "Particle swarm optimization," in Proc. IEEE International Conference on Neural Networks, Perth, Australia, 1995, vol. 4, pp. 1942-1948.

[5] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," Science, vol. 220, no. 4598, pp. 671-680, 1983.

[6] C. C. Lee, "Fuzzy logic in control systems: Fuzzy logic controller, Parts I and II," IEEE Transactions on Systems, Man, and Cybernetics, vol. 20, no. 2, pp. 404-435, 1990.

[7] C. P. Pappis and E. H. Mamdani, "A fuzzy logic controller for a traffic junction," IEEE Transactions on Systems, Man, and Cybernetics, vol. SMC-7, no. 10, pp. 707-717, 1977.

[8] F. V. Webster, "Traffic signal settings," Road Research Technical Paper No. 39, Her Majesty's Stationery Office, London, U.K., 1958.

[9] D. E. Goldberg, Genetic Algorithms in Search, Optimization and Machine Learning. Reading, MA, USA: Addison-Wesley, 1989.

[10] P. B. Hunt, D. I. Robertson, R. D. Bretherton, and R. I. Winton, "SCOOT: A traffic responsive method of coordinating signals," Transport and Road Research Laboratory, Crowthorne, U.K., TRRL Report LR 1014, 1981.

[11] A. G. Sims and K. W. Dobinson, "The Sydney coordinated adaptive traffic (SCAT) system philosophy and benefits," IEEE Transactions on Vehicular Technology, vol. 29, no. 2, pp. 130-137, 1980.

[12] B. Abdulhai, R. Pringle, and G. J. Karakoulas, "Reinforcement learning for true adaptive traffic signal control," Journal of Transportation Engineering, vol. 129, no. 3, pp. 278-285, 2003.

[13] M. Wiering, "Multi-agent reinforcement learning for traffic light control," in Proc. 17th International Conference on Machine Learning (ICML), Stanford, CA, USA, 2000, pp. 1151-1158.

[14] M. Mitchell, An Introduction to Genetic Algorithms. Cambridge, MA, USA: MIT Press, 1996.

[15] L. A. Zadeh, "Outline of a new approach to the analysis of complex systems and decision processes," IEEE Transactions on Systems, Man, and Cybernetics, vol. SMC-3, no. 1, pp. 28-44, 1973.

[16] T. J. Ross, Fuzzy Logic with Engineering Applications, 3rd ed. Chichester, U.K.: Wiley, 2010.

[17] Y. Shi and R. Eberhart, "A modified particle swarm optimizer," in Proc. IEEE International Conference on Evolutionary Computation, Anchorage, AK, USA, 1998, pp. 69-73.

[18] B. Efron and R. J. Tibshirani, An Introduction to the Bootstrap. New York, NY, USA: Chapman and Hall, 1993.

[19] Transportation Research Board, Highway Capacity Manual, 6th ed. Washington, DC, USA: National Academies Press, 2016.

[20] Python Software Foundation, "Python 3 documentation." [Online]. Available: https://docs.python.org/3/. [Accessed: Oct. 2026].

[21] NumPy Developers, "NumPy documentation." [Online]. Available: https://numpy.org/doc/. [Accessed: Oct. 2026].